

Early Warning of Hydrate Formation in Offshore Well Service Lines

XGBoost vs. TCN vs. GRU at Matched False-Alarm Budgets
3W Dataset 2.0.0 – Event 9

Mammadova, Aisel Huseynov, Shamistan Karimova, Rahima
Mammadova, Gulnur Arabova, Saida

AI Academy, Baku, Azerbaijan
DLE-AI-202, Cohort I 2026 · Track 2

September 2026

Outline

- 1 Motivation & Problem
- 2 Dataset
- 3 Method
- 4 Experimental Setup
- 5 Results
- 6 Discussion
- 7 Conclusion

Why Does This Matter?

The Hazard

Gas hydrates crystallize inside subsea service lines at high pressure & low temperature.

- Complete blockage $\Rightarrow$ well shutdown
- Costly interventions, safety risk
- Loss of production for days to weeks

The Opportunity

The **transient phase** is detectable before blockage.
An alarm at transient onset gives operators lead time to:

- Inject methanol / glycol inhibitor
- Reduce backpressure
- Warm the line

Research Question

Can a deep learning model detect the transient onset **earlier** than a feature-engineered baseline, at a **controlled false-alarm rate**?

Headline metric: Lead time (minutes from first alarm to annotated transient onset) at a target FAR of ≤ 1 alarm per 100 operating hours.

Instance counts

Split	Count
Real instances (Event 9)	57
w/ Transient phase	14
reaching blockage	3
Simulated instances	150
Normal-operation	594

Critical Fact

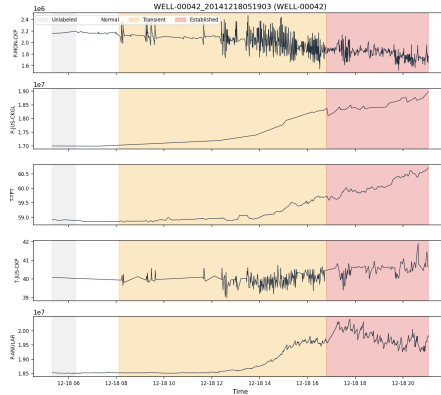
Only **14** real transient events across **7 wells**.
This is the hardest constraint in the project.

Key dataset properties

- Severity is **monotonic**: Normal $\rightarrow$ Transient $\rightarrow$ Established. Verified in all 57 instances.
- Missingness in *every* real instance (median 11.2%, max 50.4%)
- 3 instances: sensors dead for entire recording
- Hydrate wells $\cap$ Normal wells = $\emptyset$ (15 vs. 9 wells, completely disjoint)
- Per-instance normalization is **mandatory** (not cosmetic)

Preprocessing: 1 Hz $\rightarrow$ 1/30 Hz (decimation),
60-sample windows = 30 min, stride 150 s,
5-channel subset.

Annotated Sensor Trace



Colour bands: Normal / Transient / Established. Lead time measured from first alarm (dashed line) to Transient onset.

Feature Engineering — XGBoost Baseline

Multi-timescale, mask-aware features (95 total)

For each of 5 channels, extract 3 causal slices (full / last-half / last-quarter) $\times$ 6 stats (mean, std, min, max, slope, net-change) $\Rightarrow$ 90 features.

Plus: per-channel **presence fraction** (5 features) — fraction of non-missing samples.

Why presence fraction matters:

- 40.8% of XGBoost gain concentrated on 5 presence columns
- Presence-only model achieves 74% of full performance *with no sensor values at all*
- Root cause: sensor availability is well-specific & acts as a proxy for well identity

Important caveat

The baseline partly reads **instrumentation** patterns, not physical hydrate dynamics. Feature importance does not reproduce across folds ($\tau = +0.20, +0.40, -0.40$).

TCN — Temporal Convolutional Network

- 4 temporal blocks, kernel $k = 3$
- Dilations: $\{1, 2, 4, 8\}$
- Receptive field: **61 samples** > window size 60
- Causal convolutions (no future look-ahead)
- Mask concatenated at input
- 3-class softmax output

GRU — Gated Recurrent Unit

- Single-layer **unidirectional** GRU
- Hidden size: 64
- Mask concatenated at each timestep
- 3-class linear readout
- Unidirectional = causal by construction

Both models: class-weighted cross-entropy, early stopping on validation PR-AUC (patience=10), best checkpoint saved, trained on A100 GPU with AMP.

Cross-Validation Design — Preventing Leakage

Why standard CV fails here

Hydrate wells and Normal wells are **disjoint populations**.

A single StratifiedGroupKFold cannot balance both simultaneously.

Solution: two independent grouped splits

- 1 Split 7 positive-event wells by well ID
- 2 Split 9 normal wells balanced by operating hours
- 3 Pair splits to form 3 complete folds

Simulation policy

Simulated instances: **training only**. Never in validation or test.

Fold composition

Fold	Val. pos.	Test pos.	Test hrs
0	1	5	509h
1	3	5	557h
2	5	4	531h

Fold 0: **only 1** positive validation event — threshold selection is unreliable. This is a structural dataset constraint (7 wells, 3 folds), not a bug.

Evaluation Protocol — S3 Freeze

- 1 **Train**: fit model on training fold (weights frozen after)
- 2 **Validate**: sweep 200 thresholds on *validation* Normal instances, select highest threshold where $\text{FAR} \leq 1\%/h$
- 3 **Test**: apply frozen threshold to *test* fold — **no further changes**

Alarm definition

Smoothed probability (causal trailing mean) above threshold for $\geq \text{min_duration}$ seconds = one alarm onset.

Centered smoothing = test leakage. Guarded by `test_alarm.py`.

Three headline metrics

- **Event recall**: fraction of events with an alarm before transient onset
- **Lead time**: $t_{\text{transient}} - t_{\text{first alarm}}$ (minutes)
- **FAR**: alarm onsets / normal hours

XGBoost Baseline — Validation PR-AUC

Table: Validation PR-AUC (positive base rate 3.34%)

Fold	Pos. val. events	real_only	real+sim
0	1	0.0004	0.0002
1	3	0.7762	0.2111
2	5	0.0750	0.1150
mean $\pm$ sd		0.28 $\pm$ 0.43	0.11 $\pm$ 0.11

PR-AUC is not comparable across folds

Fold 0: 1 positive window in 11,260 (base rate 10^{-4})

Fold 1: 463 positive in 7,327 (base rate 6.3%)

PR-AUC rises mechanically with the base rate.

Fold variance = **47** $\times$ feature-design variance.

Simulated data did not help XGBoost. Paired difference $\text{real+sim} - \text{real_only} = -0.18 \pm 0.34$ (negative in 2 of 3 folds).

Head-to-Head: Event Recall

Table: Event Recall at the validation-selected operating point (target 1%/h; realised 7.9%/h)

Model	real_only	real+sim
XGBoost	0.233 ± 0.252	0.167 ± 0.289
TCN	0.083 ± 0.144	0.000 ± 0.000
GRU	0.000 ± 0.000	0.067 ± 0.115

Table: Mean Lead Time (minutes, detected events only)

Model	real_only	real+sim
XGBoost	43.4	209.9
TCN	89.3	—
GRU	—	34.9

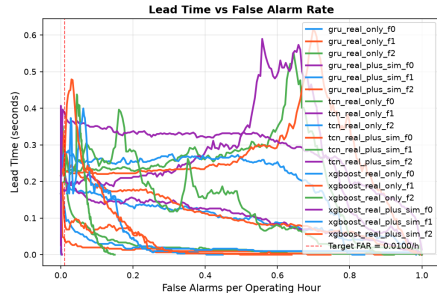
Key takeaway

XGBoost (real_only) is the **only model** with consistent non-zero recall across multiple folds.

Deep models detected events in 1 fold only (out of 3), making their lead-time numbers unreliable single observations.

Simulated data hurt every model on the headline metric (real test distribution).

Lead Time vs. False Alarm Rate Trade-off



Vertical dashed line = target operating point (1 alarm / 100 h). XGBoost (real_only) is the only model that consistently achieves non-zero lead time at or near the target FAR.

The Budget, Not the Ranking, Costs the Recall

Table: Best achievable XGBoost event recall vs. false-alarm budget

alarms/h	0.01	0.10	0.25	0.50
real_only	0.167	0.317	0.917	0.917
real+sim	0.167	0.450	0.917	0.917

Relaxing to 1 alarm/4 h moves recall 0.167 → **0.917**: a deployment trade-off, not a defect.

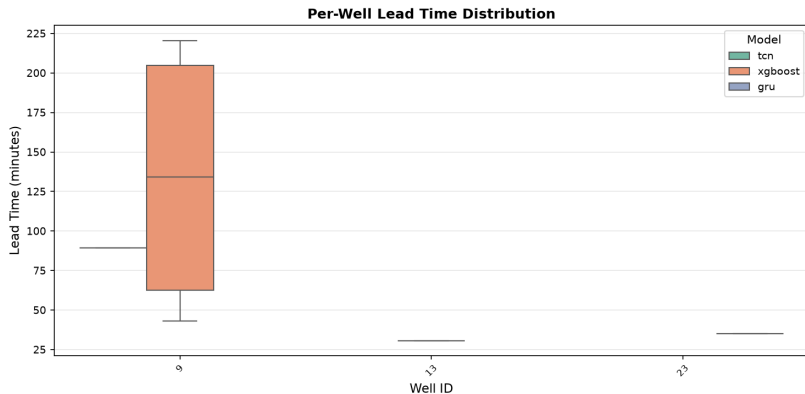
Threshold tuning is not the problem

Re-selecting the threshold *on the test fold itself* — an oracle we could never ship — changes recall in **0 of 6** XGBoost cells.

Why 1 alarm/100 h is so severe

A test fold has 523–855 Normal hours, so the budget permits only **5–9 false alarms across ≈120 Normal recordings**. The 4–5 event recordings must rank in the top few percent of *all* recordings.

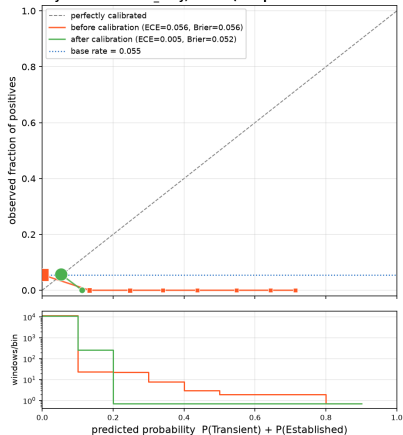
Per-Well Lead Time Distribution



High inter-well variability reflects different sensor configurations and noise profiles. With only 7 wells, no single number is representative.

XGBoost Calibration

Reliability - XGBoost real_only, fold 2 (638 positive validation windows)



Platt scaling results

- ECE: 0.041 $\rightarrow$ 0.008 (real_only)
0.054 $\rightarrow$ 0.011 (real+sim)
- Brier score: -29% in all 6 cells
- Improvement consistent across all folds and conditions

Why it matters

Threshold selection sweeps the probability axis. Without calibration the threshold doesn't transfer from validation to test.

Why Deep Models Underperformed

14 events is simply not enough

Gradient-based models with thousands of parameters need dozens of positive events to learn transition dynamics.

With 14 events across 3 folds, fold 0's single validation event makes threshold selection equivalent to random.

Why sim data hurt

- Simulated: **90.3%** positive windows
- Real: **3.34%** positive windows
- Factor of ≈ 27 distributional mismatch
- Model trains on a “hydrate-certain” world, tested on a “hydrate-rare” world
- Threshold calibration breaks

What we would do next time

- 1 SSL pretraining on 594 normal instances
- 2 Then fine-tune on labeled events
- 3 Expected: better representations without more labeled data

Sensor shortcut

XGBoost's 40.8% gain on presence-fraction columns = reading which well has which sensors, not hydrate physics.

Important caveat on generalizability.

Conclusion & Recommendations

Summary

- XGBoost (real_only): best recall **23.3%**, lead time **43.4 min**, FAR **7.9%/h** — no fold met the 1%/h budget on test ($2.6\times$, $19\times$, $2.0\times$ over)
- TCN: 8.3% recall (1 fold only), lead time 89.3 min — promising but unreliable
- GRU: 0% recall under real_only — did not generalize
- Simulated data: consistently hurts all models on real test wells

Recommendation for operators

Deploy XGBoost (real_only), but **choose the alarm budget deliberately**. At 1 alarm/100 h it recalls 17%; at 1 alarm/4 h the same model recalls **92%**. The events are detectable — the pre-registered tolerance is what excludes them.

Future directions

- 1 Acquire more labeled real Event-9 data
- 2 SSL pretraining on normal-operation archives
- 3 Multi-well domain adaptation
- 4 Physics-informed regularization

Code & data: <https://github.com/AMammedova/team-hydrate3w> (tag v1.0-final)

Team Enigma — Contributions

Member	Contributions (Code, Report, & Slides)
Mammadova, Aisel (M1)	Data analysis, annotated trace, Sec. I & III, Slides 1–3
Huseynov, Shamistan (M2)	Grouped CV design, fold report, Sec. V, Slide 4
Karimova, Rahima (M3)	95-feature extractor, XGBoost tuning & ablation, Sec. IV-A & VI-A, Slide 5
Mammadova, Gulnur (M4)	TCN & GRU architectures, A100 GPU training, Sec. II & VI-B, Slides 6–7
Arabova, Saida (M5)	Eval pipeline, alarm logic, <code>run_all.sh</code> , Abstract, Sec. IV-C, VI-C, VII, VIII, Slides 8–14

Acknowledgements & Tools

- **Dataset:** Petrobras 3W Dataset 2.0.0 (Vargas et al., 2019)
- **Compute:** AI Academy A100 GPU (80 GB VRAM)
- **Libraries:** Python 3.x, PyTorch 2.3.1, XGBoost 2.0.3, scikit-learn 1.5.0, NumPy, Pandas, Matplotlib
- **AI assistance:** Large Language Models were used to help structure evaluation boilerplate code, format $\text{\LaTeX}$ tables, and draft documentation. All AI-generated content was reviewed, tested, and validated by the team.

Thank you. Questions welcome.

<https://github.com/AMammedova/team-hydrate3w>